

Advanced Search Strategies Author: Yue (Rosy) Yan

UCS: expand node with lowest total path cost (g(n))
Depth limited search (DLS): DFS but stops at fixed depth limit
IDS: run DLS with increasing limits
Informed (Heuristic) Search Strategies - Use extra knowledge (heuristics) to guide search, h must estimate the cost at a goal node to be 0;
greedy search: expands the node with the smallest h(n) = estimate of cost from n to goal (e.g., straight-line distance)
If h(n) = g(n), will be the same as UCS
A*: expands the node with the smallest f(n).
f(n) = g(n) + h(n). g(n) = actual past cost. h(n) = estimate future cost to dest.
Complete and optimal with admissible h(n).
Admissible: $\forall n, h(n) \leq$ true cost to reach a goal from n.
Both time and space are problems for A*: to save space, use variants: IDA* or SMA*.
f f(n)=g(n), h(n)=0: acts like USC.
f f(n)=depth(n): acts like BFS. h(n)=0 or h(n)=depth(n)-g(n)
Consistent heuristics: $\forall$ edge (n→n'), h(n) ≤ cost(n,n') + h(n').
If consistent, then admissible, but not vice versa.
Dominant heuristics: given admissible heuristics, h2 dominates h1, if for all nodes n, h1(n) ≤ h2(n).
Theorem: A* using a dominant heuristic (with larger values) expands fewer nodes and is better, since it is closer to the real cost.

Local Search / Optimization Methods - not complete and not optimal, but use very low memory
Hill-Climbing Algorithm - Ignore the rest of the tree and move to the best within the neighbors of the current state.
improvements:

- Stuck on local extrema(solution found depends on initial state): random restart on different places
 - Plateaux (flat areas): allow equal v moves - allows a node to only visit k number of times (avoid infinite visit loop), and break ties by choosing smaller visited times.
 - Ridges (when single-step lowers v, but a combination of steps would climb):
 - Macro move - combine two or more actions into one move that directly increases v.
 - Limited lookahead - explore sequences of a few moves ahead and pick the sequence that eventually leads upward.
- Beam Search** - Like hill-climbing, but keep k states.
Beam Search with A*: Memory was a big problem for A*, so now we can use keep only the best k nodes in the fringe, i.e. use a priority queue of size k. But this would turn A* into not complete, and not optimal.

Simulated Annealing - To accept "bad" moves to escape local optimum. Similar to hill-climbing but selects a random successor instead of the best successor.
Cons: Choice of cooling schedule is tricky: can be slow if you cool too slowly, or poor-quality if you cool too fast. **Pros:** Escapes local optima by accepting bad moves early. Easy to implement.

Steps:
1.pick an initial T that's high enough to let you take more bad steps at first. Pick a starting solution (state) n and compute its "value" v(n) (higher is better, or lower if you're minimizing).
2. Randomly select m, one of n's successors.
3.If v(m) is better than v(n), accept the child m. Else accept m with a probability:

$$p = e^{\frac{v(new)-v(old)}{T}}$$

- Because v(m)-v(n) is negative, this makes p < 1. (p exponentially decreases with the badness of the move.)
 - When the temperature T is high, you're more likely to accept a worse move; when T is low, you almost never do.
4. After each move (or every few moves), reduce the temperature: T = $\alpha \times T$
where 0 < α < 1 (for example, $\alpha = 0.9$). This is called the cooling schedule.
5. Continue steps 2-4 until T is very low (almost zero), or you've done enough iterations. At that point, you're unlikely to accept any worse moves and you'll settle into a solution.

Genetic Algorithms - selection, crossover, mutation
Current population: S1,...,SN with fitness score: f (Si)
• Reproduction probability:

$$P_i = f(S_i) / \sum f(S_i)$$

Pros: Easy to implement. Combine: uphill tendency (based on the fitness function) & random exploration (based on crossover and mutation)
Cons: Success depends on the representation (encoding).
Fitness: value f(n) to maximize.
Operators:

- Selection: sample parents proportional to fitness.
- Crossover: swap substrings at random cut point.
- Mutation: randomly tweak bits of offspring.

Loop: Create new generation of size N via selection, crossover, mutation. Replace old population. Repeat until a good individual appears or max generations G.
Minimax MAX assumes that MIN plays optimally: proven: if MIN don't play optimally, MAX will do even better

- optimal yes, complete yes. Time and space complexity is the same with DFS.
- Time is the major problem - moves need to be chosen in a limited time

K-Nearest Neighbor (k-NN): uses the k closest neighbors and takes a majority vote (for classification) or average (for regression).

Manhattan distance:	Euclidean distance - most frequently used:
$D(A, B) = a_1 - b_1 + a_2 - b_2 + \dots$	$D(A, B) = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$

Minkowski distance - generalization of Euclidean and Manhattan:
 $D(A, B) = ((a_1 - b_1)^q + |a_2 - b_2|^q + \dots + |a_n - b_n|^q)^{1/q}$ q - positive integer

- Need for Normalization:** If one feature has a much larger numeric range than another, it will dominate the distance calculation (e.g. car weight vs cylinder count). Normalize each numeric feature to [0, 1] so all features contribute fairly.
 - Larger k smooths out noise (more neighbors vote) but may blur boundaries. A rule of thumb is $k \leq \text{sqrt}(m)$ for m examples
 - Weighted k-NN gives closer neighbors more influence (w=1/distance)
 - Curse of dimensionality** (points all look equally far apart) - "nearness" is ineffective in high dimensions $\Rightarrow$ feature selection to reduce dimensionality.

Lazy vs Eager Learning

- Lazy Learning** (k-NN): No model is built in advance, computation occurs during classification. requires full training data;
- Eager learning** (DT, Naïve Bayes, SVMs): build a model ahead of time, then classify quickly. Use rules to test the values of a single attribute.

1R Algorithm - chooses final rule based on lowest training-error attribute. e.g. 2no, 3yes $\rightarrow$ yes, errors:2/5

- Missing values can be treated as another category, or replaced by the most common value.
- Numeric features must be discretized into ranges before applying 1R. We can sort values, place breakpoints where class labels change, then merge adjacent intervals that share the same majority class to avoid overfitting.

Naive bayes algorithm:

- Assume Independence and equal importance in attributes, but assumptions are unrealistic - called naive.
- Calculate the probability of P(Yes|E) and P(No|E) and compare them, label the new example with the class of higher probability.

$$P(\text{yes} | E) = \frac{P(E_1 | \text{yes}) P(E_2 | \text{yes}) P(E_3 | \text{yes}) P(E_4 | \text{yes}) P(\text{yes})}{P(E)}$$

Problem with zero probability: if P(Ei|Yes) = 0, then the whole equation goes to 0.
Laplace Correction: for every P(Ei|Yes), +1 to numerator, +k to denominator (k = number of attribute values for the given attribute), minimal impact when the dataset is large.

handling missing values: for a new example with missing value E1, we simply ignore this attribute E1 when plugging in the formula, and only calculate P(E2|Yes) P(E3|Yes).... For a missing attribute value in the training examples, don't count that example in the denominator for that feature.
handling numeric attributes:
-P(Ei|Yes) change to: $f(x) = \frac{1}{\sigma \sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$

x = attribute value of this new example
-Mean of this attribute given class Yes: $\mu = \frac{\sum_{i=1}^n x_i}{n}$
xi= an attribute value given class Yes
-Standard dev of this attribute given class Yes: $\sigma = \sqrt{\frac{\sum_{i=1}^n (x_i - \mu)^2}{n-1}}$
n = total number of attribute values given class Yes

Cons: - Assumes conditional independence (often violated);
Solution: apply feature selection beforehand to identify and discard correlated (redundant) attributes
- Assumes normality for numeric features (often invalid);
Solution: 1. Discretize the data first, i.e. numerical $\rightarrow$ nominal attributes. 2. Use other probability density. 3.Transform the attribute using a suitable transformation into a normally distributed one (sometimes possible). 4. Use kernel density estimation which does not assume any particular distribution
Evaluating Classifiers - Generally: The larger the training data, the better the classifier. The larger the test data, the better the accuracy estimate.

examples	# assigned to class yes	# assigned to class no
# from class yes	true positives (tp)	false negatives (fn)
# from class no	false positives (fp)	true negatives (tn)

Precision: TP / (TP + FP)
Recall: TP / (TP + FN)
F1 score: 2 * P * R / (P + R)
Accuracy: Correct (TP + TN) / Total

Single holdout (train/test split). Split data into 2 non-overlapping sets: training and test (usually 2/3 and 1/3) and calculate the accuracy(proportion of correctly classified examples) and error rate(proportion of incorrectly classified examples) of the test data.
Problem: the examples in the training set might not be representative of all classes, especially rare classes $\rightarrow$ Stratification
Stratification: Split data so that each class is represented with approximately equal proportions in both data sets (training and test)
Repeated holdouts: Repeat the single holdout process many times (e.g., 10 times) on the whole dataset. (stratified if you like). Average those 10 accuracy numbers to get a more stable estimate.
Cross Validation (k-fold): Divide your dataset into K equal-size parts (folds). Repeat K times: Use K - 1 folds for training. Use the remaining 1 fold for testing. Record the accuracy on that fold. Average the K accuracy values to get the final estimate.
Leave-One-Out Cross-Validation (LOOCV): Special case of k-fold where k = n, and n is the total number of examples.

Pros: 1. The greatest possible amount of data is used for training $\Rightarrow$ increases the chance for building an accurate classifier. 2. Deterministic procedure - no random sampling is involved (no point in repeating the procedure - the same results will be obtained) Cons: 1. high computational cost $\Rightarrow$ useful for small data sets
Comparing Classifiers - Paired t-test

- Null hypothesis: The two classifiers have the same mean accuracy.
- Use the same train/test splits (e.g., k-fold). Calculate differences d, sd, σ , confidence interval Z. Check if the difference is random chance. Use paired t-test on the k paired accuracies to see see if the difference is statistically significant. If it is, conclude: "Yes, C1 is significantly better than C2 at 95% confidence."

Cost-Sensitive Evaluation: Not all misclassification errors are equal. Solution: Adjust error weights accordingly (e.g., misclassifying non-spam as spam = 10x penalty), calculate weighed A, P, F1.
Decision tree: Hill climbing with information gain as an evaluation function. Information gained = entropy - entropy after split. Entropy: smaller the cleaner.

For our example: weather data - 9 yes and 5 no examples:
 $H(S) = -P_{\text{yes}} \log_2 P_{\text{yes}} - P_{\text{no}} \log_2 P_{\text{no}} = I(P_{\text{yes}}, P_{\text{no}}) = I(\frac{9}{14}, \frac{5}{14}) = -\frac{9}{14} \log_2 \frac{9}{14} - \frac{5}{14} \log_2 \frac{5}{14} = 0.940 \text{ bits}$

k-means: 1. Choose k examples as the initial centroids of the clusters.
2. For each example, find the closest centroid to it, and assign it.
3. At the end of each epoch: Re-compute the centroid of the clusters. Check if the stopping criterion is satisfied: centroids change < 1%.

Nearest-Neighbor Clustering: 1. Take first point t1 to form cluster K1={t1}.
2. For each new point ti, compute the distance from ti to every point in each existing cluster Kj and find the minimum distance d_min. This uses single-link (MIN) distance between ti and cluster Kj. 3. Threshold t: If d_min ≤ t, add ti to that cluster; otherwise, create a new cluster containing ti.
Hierarchical Clustering Algorithms: Agglomerative (bottom-up) - merges clusters: start from singleton cluster, merge two clusters if their distance is ≤ a threshold t. t is incremented in each level (level closer to root will allow greater threshold). Merge until only one cluster.

Log O is not defined, so we use log O = 0
Information Gain of an attribute A on dataset S is how much the entropv decreases if you split S by A:
 $\text{Gain}(S, A) = H(S) - H(S|A)$
Example(Weather Data, splitting on "Outlook"):
Parent Entropy H(S): For 9 "yes" and 5 "no," H(S) ≈ 0.940 bits.
Outlook = Sunny: (2 "yes," 3 "no") $\rightarrow$ entropy ~ 0.971 bits.
Outlook = Overcast: (4 "yes," 0 "no") $\rightarrow$ entropy = 0 bits.
Outlook = Rain: (3 "yes," 2 "no") $\rightarrow$ entropy ~ 0.971 bits.
Weighted sum H(S|Outlook)=(5/14) × 0.971+(4/14) × 0+(5/14) × 0.971 ≈ 0.639 bits.
Information Gain: 0.940-0.639=0.247 bits.

Tree Construction Stop if 1) All examples have the same class. $\rightarrow$ assign such class 2) All examples have identical attribute values but differing classes (label noise) $\rightarrow$ assign majority class. 3) The subset is empty $\rightarrow$ assign majority class from the parent set.

Overfitting: the error on training data is very small but the error on new data(test data) is high. **Reasons:** 1. Training data is too small $\Rightarrow$ not enough representative examples to build a model that can generalize well on new data.
2.Noise in training data (incorrectly classified examples) $\Rightarrow$ DT learns them but they're incorrect $\Rightarrow$ will incorrectly classify new examples. **How to avoid?**

greedy to choose best subtree to be replaced with leaf node:

- Pre-Pruning:** Stop growing the tree earlier, before it reaches the point where it perfectly classifies the training data. Using e.g. Minimum number of instances per leaf node, Maximum tree depth. Minimum gain threshold. Goal: prevent the tree from modeling noise or overly small partitions. Con: may stop too early and underfitting (must decide a good stopping rule)
- Post-Pruning by Subtree Replacement:** Grows the full tree, then **prune** it. Uses a separate **validation set** (distinct from training and test sets) to assess pruning impact. If replacing a subtree with a leaf improves or with same validation accuracy, perform pruning. Iterate until no further improvement is possible. **Pros:** reduces tree size and avoids memorizing noise.
- Post-Pruning by Rule pruning:** Convert a complete DT to a set of rules, then **prune** it. Iteratively prune rules by removing preconditions that are not harmful. Sort the pruned rules by their validation-set accuracy and use that ordered rule list for classification of new instances. **Pros:** Rule sets can be more readable than large trees.

Handling Numeric(continuous) Attributes - Decision trees require nominal splits. Discretization:

- Sort training examples by the numeric attribute's value.
- Identify adjacent pairs where the class label changes (e.g., one example is "yes," the next is "no").
- For each such adjacent boundary, consider a candidate split point halfway between the two numeric values (between 70 and 71 is 70.5).
- Evaluate the information gain at each candidate threshold. Choose the threshold that yields the highest gain.

Highly branching attributes (ID number) are more likely to create a pure set (because they are extremely messy, any split would clean it a bit), and thus will have the highest information gain. But more likely lead to overfitting. Introduce Gain Ratio to penalize:

$$\text{GainRatio}(S|A) = \frac{\text{Gain}(S|A)}{\text{SplitInformation}(S|A)}$$

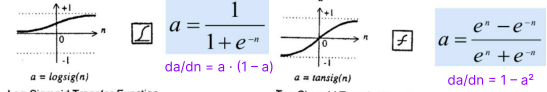
$\text{splitInformation}(S|A) = -\sum_{i=1}^{|S|} \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$

$$\text{SplitInformation}(S|Idcode) = -\frac{1}{14} \log_2 \frac{1}{14} * 14 = 3.807 \text{ bits}$$
$$\text{GainRatio}(S|IdCode) = \frac{0.940}{3.807} = 0.246 \text{ bits}$$

Sometimes different attributes have different costs, and we want DT to branch on attributes with lower cost first, so we can pay low cost when classifying new examples. Solution: include Cost function when calculating Gain(S|A) by penalizing attributes with high costs: $\text{Nunez, where } w \in [0,1] \text{ determines cost importance}$

$$\frac{\text{Gain}^2(S|A)}{\text{Cost}(A)}$$
$$\frac{2^{\text{Gain}(S|A)} - 1}{(\text{Cost}(A) + 1)^w}$$

Neural Networks
Perceptrons (complete if linearly separable, not optimal)
You have a training dataset of m examples. Each example is a pair (p, t) where p=[p1, p2, ..., pn] is an input vector, and t∈{0,1} is the target correct label. A weight vector is orthogonal to the decision boundary that separates region 0 and region 1.
• Adjustment to the weight vector is rotating the decision boundary, adjustment to bias term is shifting the db.
Start with small random weights w1,w2,...,wn and bias b.
Sometimes we start them all at zero, but often a tiny random number is used. Compute a: net input: n = w1p1 + w2p2 + b.
step function: $a = f(n) = \begin{cases} 1 & \text{if } n \geq 0 \\ 0 & \text{if } n < 0 \end{cases}$
Compare a with target t. Define the error e = t - a.
Update the weight vector and bias: wi += e * pi, b += e
If e=+1, you push the weights toward p. If e=-1, you pull them back by p. If e=0, you don't change anything.
One "epoch" means you have gone through all training examples once, updating weights on each mistake.
Stop condition: all examples after an epoch are now correctly classified (i.e., a=t for every example). Or, stop after certain number of epochs.
Cons: can only solve problem with binary class.
Backpropagation(hill climbing the deepest descent in the weight space). Instead of step function, use activation function(need to be continuous and differentiable):



Initialization of Weights: Uniform random in [-1, +1], so each neuron learns independently.
Input Neurons: Corresponding to attributes. for 4 numeric attributes (Iris), we can have 4 input neurons. Categorical attribute "Weather = {Sunny, Overcast, Rainy}" can have 3 neurons by one-hot encoding.
Output Neurons: Corresponding to class. A binary classification can have 1 output neuron interpreted as "probability of class 1". Or, use one neuron per class, toggle 1/0 for true/false.
Hidden Neurons: heuristic = $\frac{(\text{number of inputs}) + (\text{number of outputs})}{2}$
Too many hidden neurons can lead to overfitting; too few can lead to underfitting. Often one tries a small network first, then gradually increases size until performance on a validation set stops improving.

Universal Approximation Theorem (Cybenko): a single hidden layer with enough neurons can approximate any continuous function. Two hidden layers can even approximate any discontinuous functions.
Forwarding in a neuron network (layer: input i → j → k output)
 $net_j = \sum_i w_{ij} a_i + b_j$ wij = the weight from neuron i to j
The value at each node is calculated by: $a_j = f(net_j) = \frac{1}{1 + e^{-(\sum_i w_{ij} a_i + b_j)}}$
Error Function (a = o): For classification, we measure how far off the network's outputs are from the desired targets. **Sum of Squared Errors (SSE):** $E(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^{n_{out}} (d_i - a_i)^2$ $E_{total} = \sum_{p=1}^M E^p$

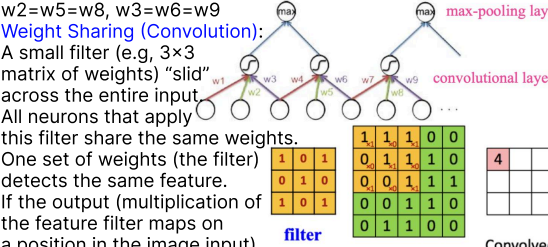
di is the target value for output neuron i, ai is the actual activation produced by the neuron i.
Over an entire training set of M examples, you can define total error as E_total by adding each example's error.
Steepest Gradient Descent: Goal: Find the set of all weights (and biases) that minimize the error E. The weight vector lives in a high-dimensional space.
If using **batch** mode, accumulate updates over all examples, then do a single weight update at the end of the epoch.
If using **stochastic** mode, do updates once per example. usually faster and can escape local optima than batch.

Weight update rule: $w_{ij} \leftarrow w_{ij} - \eta \frac{\partial E}{\partial w_{ij}} = w_{ij} - \eta \delta_j a_i$
η=0 is learning rate
for output neuron j: $\delta_j = (d_j - a_j) f'(net_j)$
for hidden neuron j: $\delta_j = f'(net_j) \sum_{k \in (\text{next layer})} w_{jk} \delta_k$
the summation sums every neurons in the one next layer (next is closer to the output layer).
Stop condition: run until k number of epochs, or E_total below a threshold. (validation set can early stop to avoid overfitting).
Learning Rate (small constant): Too small → Convergence is extremely slow (tiny steps down the slope), Too large → may overshoot minima, causing oscillations or divergence (weights keep jumping past the minimum).
Momentum Term: The weight change in the current iteration would be impacted by the weight change from the last iteration. $\Delta w_{pq}(t) = \eta \delta_q o_p + \mu \Delta w_{pq}(t-1)$ where $\mu \in [0, 1]$ is the momentum coefficient (often in 0.6-0.9), and $\Delta w_{pq}(t)$ is the update at iteration t. Then $w_{pq}(t) = w_{pq}(t-1) + \Delta w_{pq}(t)$.
If a weight change in the last iteration was large (a steep slope), we keep part of that momentum in the current iteration to move faster. If the gradient changes sign, the momentum term helps blunt the reversal, smoothing oscillations.

Backpropagation problems:
1. The vanishing gradient problem – the weight changes for the lower levels are very small; these layers learn slower than the higher hidden layers. 2. training is slow (requires many epochs)
3. network fully connected, too many parameters to adjust
4. The weights are initialized randomly (may get stuck in local optimum or plateau) 5. Require a large dataset of labeled data (sometimes not available) 6. Require feature-engineering, selecting useful features 7. the probability to find a poor local minimum increases as the number of hidden layers increases.
Autoencoder NN can solve problem 2, 4, 5, 6, 7 (the output is trained to match the input: input and output value are the same, one hidden layer of smaller number of neurons: can be use for image and data compression, and encryption(weight x-h encrypt, weight h-y decrypt)
Stacked Autoencoders: Each layer of the network uses the trained hidden layer before it as input. Can learn hierarchical features in an unsupervised way - training this second autoencoder forces it to learn compressed representations of the first layer's features. The training of a deep NN has 3 steps:

1. Pre-training step: using stacked autoencoder, train a sequence of autoencoders, 1 for each layer (unsupervised)
2. Fine-tuning step one: Train the last layer using backpropagation (supervised) 3. Fine-tuning step two: train the whole network using backpropagation (supervised)
Purpose of pre-training: (1) obtain better weight initializations that encode meaningful structure (no random initialization that could lead to poor local optimum) (2) exploit unlabeled data to learn hierarchical features.

Convolutional Neural Networks (CNNs) can solve problem 3: Designed to process image data (and other grid-like data) while drastically reducing parameters. Each neuron in a convolutional layer only connects to a small, localized patch of the previous layer (called the receptive field) instead of the entire layer, cutting down the number of weights. w1=w4=w7, w2=w5=w8, w3=w6=w9
Weight Sharing (Convolution): A small filter (e.g. 3x3 matrix of weights) "slid" across the entire input. All neurons that apply this filter share the same weights. One set of weights (the filter) detects the same feature. If the output (multiplication of the feature filter maps on a position in the image input) has high value, feature is detected, if low value, feature is not detected. Can have more than one filter to detect different features. Filters are not pre-designed, they are learned.



Pooling: After convolution, a pooling layer (often max-pooling) takes the maximum over small regions (e.g., 2x2). This provides translation invariance (small shifts in the input don't affect the output) and reduces dimensionality. Help images and sounds (translation is one of the major sources of distortion).
Local Contrast Normalization (LCN): can be applied after pooling; It normalizes the output of each max-pooling neuron by subtracting the mean of their incoming neurons and dividing by the standard deviation of these neurons. allows for brightness invariance, useful in image recognition.
Modified backpropagation with CNN: input → convolution layer → pooling layer → convolution layer → pooling layer → fully connected layer → fully connected layer → output.
Dropout: In fully connected layers toward the end of a CNN, randomly "drop" (set to zero) a fraction of neurons during training to prevent overfitting. This forces the network not to rely heavily on any single neuron (to be more robust to noise).
Support Vector Machines is a method for classifying data into two groups. It finds a line that separates the two classes with the largest possible gap between them, generalizes well to new data. The decision boundary is defined by a subset of the training vectors called support vectors (data points closest to decision boundary) → resistant to overfitting.
moving non-support vectors does not affect the decision boundary.

For each training example (xi,yi), where xi is a feature vector and yi is either +1 or -1, A hyperplane in d-dimensions can be written as w·x+b=0.
hard-margin constraint: $y_i (w \cdot x_i + b) \geq 1 \quad \forall i$
every point on the correct side + at least one margin unit away
soft-margin constraint: $\begin{cases} \sum_{i=1}^N \lambda_i y_i = 0, \\ 0 \leq \lambda_i \leq C, \quad \forall i \end{cases}$

where λi are Lagrange multipliers and C controls how much misclassification is allowed. A large C makes the classifier try harder to avoid errors (narrower margin), while a small C allows more misclassifications but yields a wider margin.
Optimization Problem: Maximizing the margin is maximizing $\frac{1}{2} \|w\|^2$ given the constraints. This is a quadratic-programming (QP) problem: Only those λi associated with support vectors end up nonzero.
 $w = \sum_{i \in SV} \lambda_i y_i x_i$ where "SV" is the set of support vectors.

- During training, compute dot products between pairs of training vectors;
- During testing – between the testing example and the support vectors.

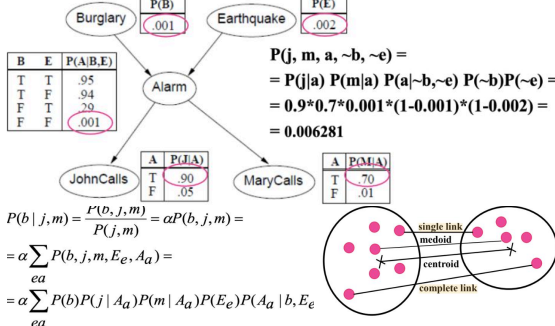
From Linear to Nonlinear: **The Kernel Trick:** Sometimes data are not linearly separable in the original feature space. The trick is to map each point x into a higher-dimensional space via a function Φ(x), where they might become linearly separable. We would need to recompute dot products Φ(xi)·Φ(xj) in that high-dimensional space. Doing so directly becomes computationally expensive if the new space is large or infinite. A kernel function K(xi,xj) is any function that computes Φ(xi)·Φ(xj) without explicitly finding Φ(x) (does not need to do xi→Φ(xi), xj→Φ(xj), Φ(xi)·Φ(xj), but instead do xi,xj→Φ(xi)·Φ(xj)). This is valid only for kernels that satisfy Mercer's condition (i.e., there exists some mapping Φ for which K is a dot product).

Training with kernel function: $w = \sum_{i=1}^n \lambda_i y_i \Phi(x_i)$
The optimization finds λi values under the constraint $0 \leq \lambda_i \leq C$ (or just $\lambda_i \geq 0$ for hard-margin).
Classification with kernel function: To classify a new point z, compute: $f(z) = \sum_{i \in SV} \lambda_i y_i K(x_i, z) + b$
then predict class +1 if f(z)>0, else -1. Because only support vectors appear, evaluation is efficient once training is done.
Ensembles: bagging: create M bootstraps from original data. (sampling with replacement, same number of examples as original, so some examples repeats). On average, 63% of the examples in the original will also appear in the bootstrap. Build classifier on each bootstrap. Take a majority vote.

Boosting(e.g., AdaBoost): Train classifiers sequentially. weighted majority vote. **AdaBoost:** training set T contains 10 examples {x1, x2, ..., x10}, each example xi has a class yi. p(xi) = 1/m = 1/10. Using this probability to draw 10 examples with replacement to form T1 and build C1. Test C1 on T, calculate the error rate ε1: $\epsilon_1 = \sum_{i=1}^{10} p_1(x_i) \cdot e_1(x_i) = 0.3$
e1(xi) = 0 if correctly classified, e1(xi) = 1 if incorrect.
Then, a term B is calculated for weight modification: $\beta_1 = \epsilon_1 / (1 - \epsilon_1) = 0.43$ and probability of correctly classified examples are modified as $p(x_i) = p(x_i) \cdot \beta_1$
Normalization: dividing each probability by the sum of all probabilities so that all probability sums to be 1.
Boosting Theorem: Even if each base classifier is just slightly better than random (a "weak learner"), AdaBoost can combine enough of them to perfectly fit the training data.
Random forest(use Bagging to build a forest of DTs on random k features, prove to resist DT overfitting): Build each DT on a bootstrap sample without pruning. At each split, randomly choose k features from the m total and pick the best split among those k. To classify a new example, send it down each of the M trees and take a majority vote of their predictions.
Choosing k: commonly k=log_2(m). Smaller k increases diversity (less correlation) but may weaken each tree; larger k makes trees stronger but more correlated.

Probabilistic Reasoning.
P(A, B) = P(A and B) P(A ∨ B) = P(A) + P(B) - P(A, B).
 $P(a, b, c, d) = P(a | b, c, d) P(b | c, d) P(c | d) P(d)$
 $P(X) = P(X | Y) P(Y) + P(X | \neg Y) P(\neg Y)$
 $P(b | j, m) = \frac{P(b, j, m)}{P(j, m)} P(a | b, \sim b, \sim c) = \frac{P(B | A)}{P(B)}$

A and B are conditional independent given C:
 $P(A, B | C) = P(A | C) P(B | C) \iff P(A | B, C) = P(A | C)$



$P(b | j, m) = \frac{P(b, j, m)}{P(j, m)} = \alpha P(b, j, m)$
 $= \alpha \sum_{ea} P(b, j, m, E_e, A_e) = \alpha \sum_{ea} P(b) P(j | A_e) P(m | A_e) P(E_e) P(A_e | b, E_e)$
Unsupervised Learning (Clustering) - High cohesion (i.e. high similarity within the cluster), High separation (i.e. low similarity between the clusters). **Distance between clusters:** $\sum_{i=1}^N P_i$
Centroid(means) - for a cluster {p1, p2, ..., pn}: $C = \frac{1}{N}$
Medoid - the centrally located data point in a cluster.
Average link - the average pairwise distance between elements from each cluster: $\frac{\sum_{i \in K_i} \sum_{j \in K_j} \text{dist}(x_i, x_j)}{|K_i| |K_j|}$
Davies-Bouldin (DB) index: A heuristic measure of the quality of the clustering. k: number of clusters. ci = centroid of cluster i. dist(x, ci) = distance between the centroids of cluster i and j. **Smaller DB indicates better clustering.**
K-means: an optimization problem minimizing the sum-squared error (SSE) of Euclidean distance: $SSE = \sum_{i=1}^k \sum_{x \in K_i} \text{dist}(x, c_i)^2$
not optimal (can be local minimum)
Data should be normalized,
Nominal data need to be converted into numeric.
Typical values of k: 2 to 10. Converges quickly(The number of epochs is typically much smaller than the number of points) but choice of initial centroid is sensitive, not sensitive to order of input examples. Does not work well with outliers or when desired clusters are not round-shape.
Single-linked linkage: Pros: Captures elongated clusters. **Cons:** "chaining effect": two clusters that should be separate can get stitched together by a series of intermediate points. Sensitive to noise: a single noisy point that lies between two clusters can cause them to merge prematurely.
Complete-linked linkage: can avoid "chaining effect" in single-link. Not sensitive to noise.